AI/ML Track

Interview Preparation Series • Professional Edition

Transformers & LLMs

04

Compiled with LaTeX • June 30, 2026

harsha.somisetty01@gmail.com

In This Edition

> The Transformer replaced recurrence with attention, and that single architectural decision unlocked the scaling laws that produced GPT-4, Claude, Gemini, and every frontier AI system in production today.

1 Overview --- What it is

A **Transformer** is a neural network architecture built entirely on attention mechanisms — no recurrence, no convolution. Introduced by Vaswani et al. in 2017 ("Attention Is All You Need"), it processes entire sequences in parallel, allowing massive parallelism on modern GPUs/TPUs.

A **Large Language Model (LLM)** is a Transformer trained at scale (billions of parameters, trillions of tokens) to model the probability distribution of text. The emergent result is a system that can reason, code, translate, summarize, answer questions, and engage in open-ended dialogue.

1.1 What makes them special

- **Attention is global**: every token can directly attend to every other token in the context
- **Parallelizable training**: unlike RNNs, all positions are processed simultaneously
- **Scale works**: more parameters + more data + more compute = reliably better models (scaling laws)
- **Transfer learning**: pretrain once on internet-scale data, fine-tune cheaply for any task

1.2 Key numbers to know

Model	Parameters	Context Window	Year
BERT-base	110M	512 tokens	2018
GPT-2	1.5B	1024 tokens	2019
GPT-3	175B	2048 tokens	2020
PaLM	540B	2048 tokens	2022
GPT-4 (est.)	~1.8T (MoE)	128K tokens	2023
Llama 3.1	405B	128K tokens	2024
Claude 3.5 Sonnet	Unknown	200K tokens	2024
Gemini 1.5 Pro	Unknown	1M tokens	2024

2 Why It Exists (the path from RNNs → "Attention Is All You Need" → GPT scale)

2.1 The problem with RNNs and LSTMs

Before Transformers, sequence modeling relied on **Recurrent Neural Networks (RNNs)** and their improved variant **LSTMs** (Long Short-Term Memory):

```
RNN: h_t = tanh(W_h * h_{t-1} + W_x * x_t + b)

Input:  [w1] → [w2] → [w3] → [w4] → [w5]
          ↓      ↓      ↓      ↓      ↓
Hidden: [h1] → [h2] → [h3] → [h4] → [h5]
```

Fatal flaws:

1. **Sequential bottleneck**: h_t depends on h_{t-1} — cannot parallelize across time steps
2. **Vanishing gradient**: gradients decay exponentially over long sequences; LSTM helped but didn't solve it
3. **Fixed-size bottleneck**: the entire past is compressed into a single hidden vector h_t
4. **Long-range forgetting**: "The cat, which sat on the mat, was..." → by the time you reach "was", the model has weakened signal from "cat"

2.2 Seq2Seq + Bahdanau Attention (2014-2015) --- the precursor

Bahdanau et al. added an **attention mechanism** to seq2seq models: the decoder, when generating token t, could look back at ALL encoder hidden states and form a weighted sum. This was a breakthrough — but attention was still bolted onto recurrent models.

```python
Attention score: e_{ij} = alignment(s_{i-1}, h_j)
                 α_{ij} = softmax(e_{ij})
Context vector: c_i = Σ_j α_{ij} * h_j
```

2.3 "Attention Is All You Need" (2017) --- the revolution

Vaswani et al. asked: **what if we use ONLY attention and remove recurrence entirely?**

Key insight: if attention can let any position see any other position, we don't need sequential processing at all. Just stack attention layers with position information baked in.

Result: the Transformer.

Why this was transformative:

- Training became fully parallelizable → 10-100x faster
- No gradient vanishing over sequence length
- Any token directly attends to any other token (O(1) path length vs O(n) for RNN)
- Scales cleanly with more compute

2.4 The GPT arc (2018-present)

```
2018: GPT-1    (117M params)  → pretraining + fine-tuning works
2018: BERT     (340M params)  → bidirectional pretraining dominates NLU
2019: GPT-2    (1.5B params)  → "too dangerous to release" — language model can write
2020: GPT-3    (175B params)  → few-shot learning, emergent abilities
2021: Codex    (12B params)   → GitHub Copilot launches
2022: ChatGPT  (GPT-3.5+RLHF) → 100M users in 60 days
2023: GPT-4    (~1.8T MoE?)   → multimodal, bar exam top 10%
2024: Claude/Gemini/Llama 3 → long context, open weights, SOTA
```

The key realization: the pretraining objective (predict next token) is an *extremely* rich self-supervised signal. Given enough data and compute, models develop internal representations of grammar, facts, reasoning, code, and more — without any labeled data.

3 Why FAANG & AI Labs Care

3.1 OpenAI

- Core product IS the LLM (GPT-4, o1, o3). Revenue: API + ChatGPT subscriptions
- Research frontier: RLHF, Constitutional AI concepts, reasoning (o1/o3 chain-of-thought)
- Interview focus: attention internals, training at scale, alignment

3.2 Anthropic

- Safety-focused frontier lab. Claude family (Haiku/Sonnet/Opus)
- Pioneered Constitutional AI (RLHF with a "constitution" of principles instead of human raters)
- Research: mechanistic interpretability, long context (200K), computer use
- Interview focus: RLHF vs DPO, safety, interpretability, scaling

3.3 Google DeepMind

- Gemini family (Ultra/Pro/Flash/Nano), Bard/Gemini app
- TPU infrastructure, PaLM/Chinchilla scaling laws, AlphaCode
- Research: Chinchilla (optimal compute allocation), Flash Attention, speculative decoding
- Interview focus: distributed training (TP/PP/DP), TPU design, efficient inference

3.4 Meta AI

- Llama family (open weights — Llama 2, 3, 3.1 up to 405B)
- FAISS for vector search, PyTorch foundation, LLaMA → LLaMA Guard
- Research: LoRA (Microsoft/Meta), quantization, RoPE positional encodings
- Interview focus: open source ecosystem, PEFT methods, inference optimization

3.5 Microsoft

- Heavy Azure OpenAI investment, GitHub Copilot, Bing Chat
- Research: LoRA (Hu et al. 2021), DeepSpeed (ZeRO optimizer), GPTQ quantization
- Interview focus: serving LLMs at scale, LoRA, product integration

3.6 Amazon (AWS)

- Bedrock platform (multi-model API), Titan models, Alexa LLM
- Focus: enterprise deployment, cost optimization, retrieval-augmented generation (RAG)
- Interview focus: inference economics, RAG pipelines, MLOps, serving latency

Bottom line: every major tech company's AI strategy depends on Transformers. Understanding them deeply is table stakes for any AI/ML role at FAANG level.

4 Core Concepts

4.1 Tokenization

Before a model sees text, it must be converted to discrete tokens (integers).

Why not characters or words?

Unit	Vocabulary size	Problem
Characters	~256	Sequences too long; no semantic meaning
Words	300K+	Rare words missing; morphology ignored
Subwords	30K-100K	Sweet spot — handles OOV, reasonable length

Byte-Pair Encoding (BPE)

The dominant approach (GPT family, RoBERTa):

1. Start: each character is a token

2. Count all consecutive pair frequencies
3. Merge the most frequent pair into a new token
4. Repeat until vocabulary size reached

```
Corpus: "low lower lowest"

Start:  l o w , l o w e r , l o w e s t
Step 1: "lo" is frequent → merge: lo w , lo w e r , lo w e s t
Step 2: "low" is frequent → merge: low , low e r , low e s t
Step 3: "lower" is frequent → merge: low , lower , low e s t
...
```

Result: common words become single tokens; rare words split into meaningful pieces. `"ChatGPT"` → `["Chat", "G", "PT"]` (approximately)

WordPiece (BERT)

Similar to BPE but maximizes likelihood instead of frequency. Uses ## prefix for continuation pieces: `"playing"` → `["play", "##ing"]`

SentencePiece (T5, Llama)

Language-agnostic, treats input as raw bytes/unicode. No pre-tokenization required. Supports unigram language model or BPE.

Key interview point: tokenization affects everything — cost (you pay per token), context window utilization, and model behavior on numbers/code. Numbers are often split digit-by-digit: `"12345"` → `["1", "2", "3", "4", "5"]` — this is why LLMs struggle with arithmetic.

4.2 Embeddings

After tokenization, each token ID is mapped to a dense vector via a learned **embedding matrix** E ∈ ℝ^{V×d_model}.

```
Token: "cat" → ID: 5023
Embedding: E[5023] = [0.23, -0.14, 0.87, ..., 0.02]  # d_model dimensions
```

Properties:

- Semantic similarity encoded in vector space
- `king - man + woman ≈ queen` (classic example)
- These embeddings are learned during pretraining and encode rich linguistic knowledge
- Input to the Transformer is: token embeddings + positional encodings

4.3 Self-Attention --- The Core Mechanism

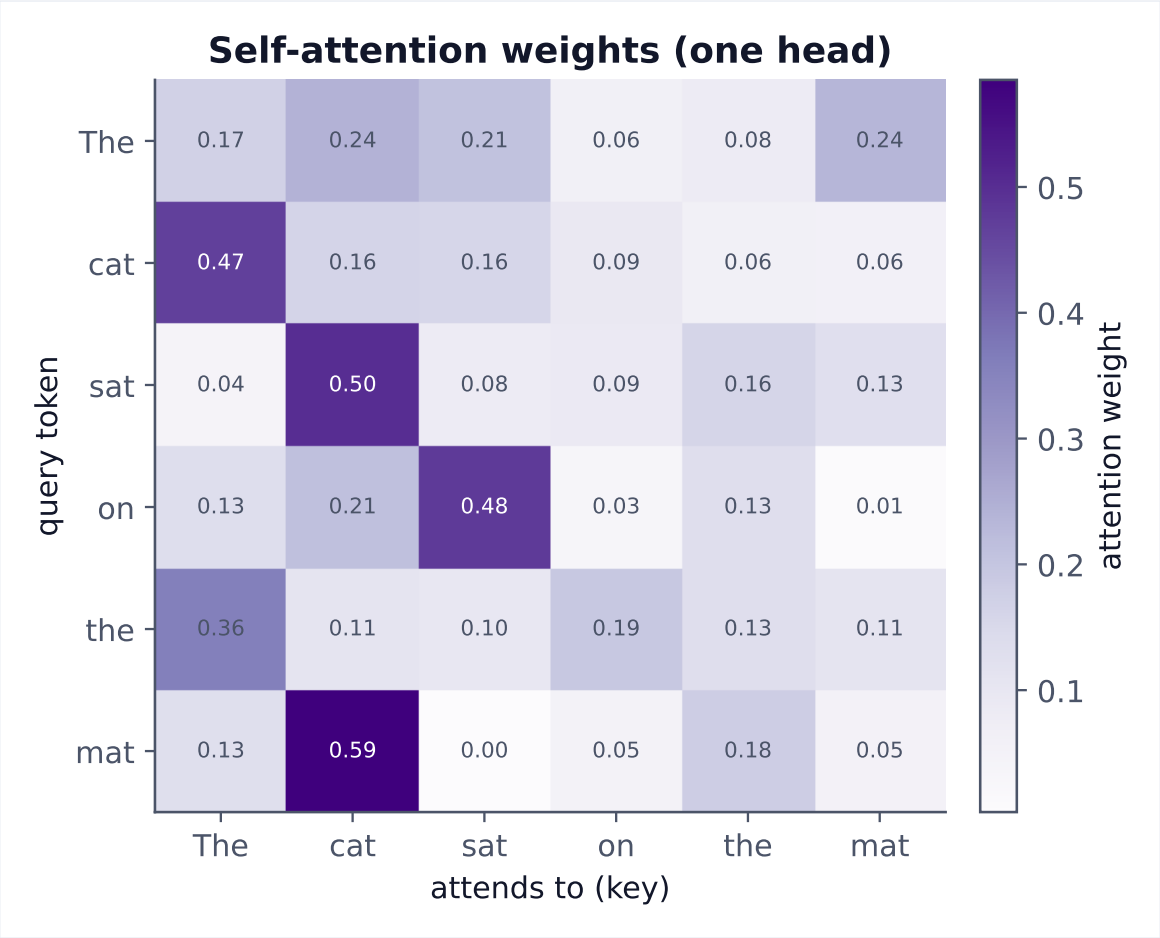


Figure 1. Each query token forms a weighted average over all tokens via softmax($QK^T/\sqrt{d}$). Rows sum to 1 — the model learns which context positions matter.

The fundamental idea: for each token, compute a weighted sum of all token representations, where weights reflect relevance.

Query, Key, Value (Q, K, V)

Every token produces three vectors via learned linear projections:

```python
Q = X W_Q    (Query:  "What am I looking for?")
K = X W_K    (Key:    "What do I offer/label myself as?")
V = X W_V    (Value:  "What is my actual content?")

Where X ∈ ℝ^{n×d_model}, W_Q, W_K, W_V ∈ ℝ^{d_model×d_k}
```

Intuition (from the newsroom analogy):

- **Q (Query)**: the question an editor is asking (”who is the subject of this sentence?”)
- **K (Key)**: each word's label/tag on the board (”I am a proper noun”, ”I am a verb”)
- **V (Value)**: each word's actual content (the word's semantic meaning to pass along)

Scaled Dot-Product Attention

```
                          QK^T
Attention(Q, K, V) = softmax(------) V
                          √d_k
```

Step-by-step:

```python
Step 1: Compute raw scores (compatibility between each query and all keys)
        scores = Q · K^T          shape: (n_tokens × n_tokens)

Step 2: Scale to prevent softmax saturation
        scores = scores / sqrt(d_k)

Step 3: Apply softmax to get attention weights (probabilities)
        weights = softmax(scores)   shape: (n_tokens × n_tokens)
        # Each row sums to 1.0

Step 4: Weighted sum of values
        output = weights · V        shape: (n_tokens × d_k)
```

Why $\sqrt{d\_k}$ scaling? Without it, for large d_k, dot products grow large in magnitude, pushing softmax into saturation regions where gradients vanish. Dividing by $\sqrt{d\_k}$ keeps the variance of the dot product ~1.

```python
import numpy as np

def scaled_dot_product_attention(Q, K, V, mask=None):
    """
    Q: (batch, heads, seq_len, d_k)
    K: (batch, heads, seq_len, d_k)
    V: (batch, heads, seq_len, d_v)
    """
    d_k = Q.shape[-1]

    # (batch, heads, seq_len, seq_len)
    scores = np.matmul(Q, K.transpose(0, 1, 3, 2)) / np.sqrt(d_k)

    if mask is not None:
        scores = scores + mask  # mask = -inf for positions to ignore

    weights = np.exp(scores) / np.sum(np.exp(scores), axis=-1, keepdims=True)

    # (batch, heads, seq_len, d_v)
    output = np.matmul(weights, V)
    return output, weights
```

Attention Complexity

Dimension	Cost
Compute	$O(n^2 \cdot d)$ per layer — quadratic in sequence length
Memory	$O(n^2)$ for attention matrix — the KV cache grows with context

This quadratic bottleneck is why long context is hard and why Flash Attention, linear atten-

tion, and sparse attention matter.

4.4 Multi-Head Attention

Run self-attention h times in parallel, each with different learned projections:

```python
head_i = Attention(Q W_Q^i, K W_K^i, V W_V^i)

MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W_O

where W_Q^i ∈ ℝ^{d_model×d_k}, d_k = d_model / h
```

Why multiple heads?

- Each head can specialize in different relationships
- Head 1 might focus on syntactic dependencies (subject→verb)
- Head 2 might focus on coreference (pronoun→antecedent)
- Head 3 might focus on semantic similarity
- More expressive than a single attention computation
- In practice, different heads empirically learn different linguistic features

Key numbers: GPT-3 uses h=96 heads, d_model=12288, d_k=128 per head.

4.5 Positional Encoding

Attention is **permutation-invariant** — without position information, "dog bites man" = "man bites dog". We inject position information by adding positional encodings to token embeddings.

Sinusoidal (original Transformer)

```
PE(pos, 2i)   = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

- Different frequencies for different dimensions
- Relative positions can be computed from these (key property)
- Generalizes to sequences longer than training length (theoretically)

Learned Absolute Positional Embeddings (GPT-2, BERT)

Simply learn a vector for each position 0..max_len. Simple but doesn't extrapolate.

Rotary Positional Embedding (RoPE) --- Llama, GPT-NeoX

Encodes position by rotating Q and K vectors. Relative position naturally falls out of the dot product. Better length generalization.

ALiBi (PaLM, BLOOM)

Add a linear bias to attention scores based on distance: score -= m * |i - j|. Simple, extrapolates well, no learned parameters.

4.6 The Full Transformer Block

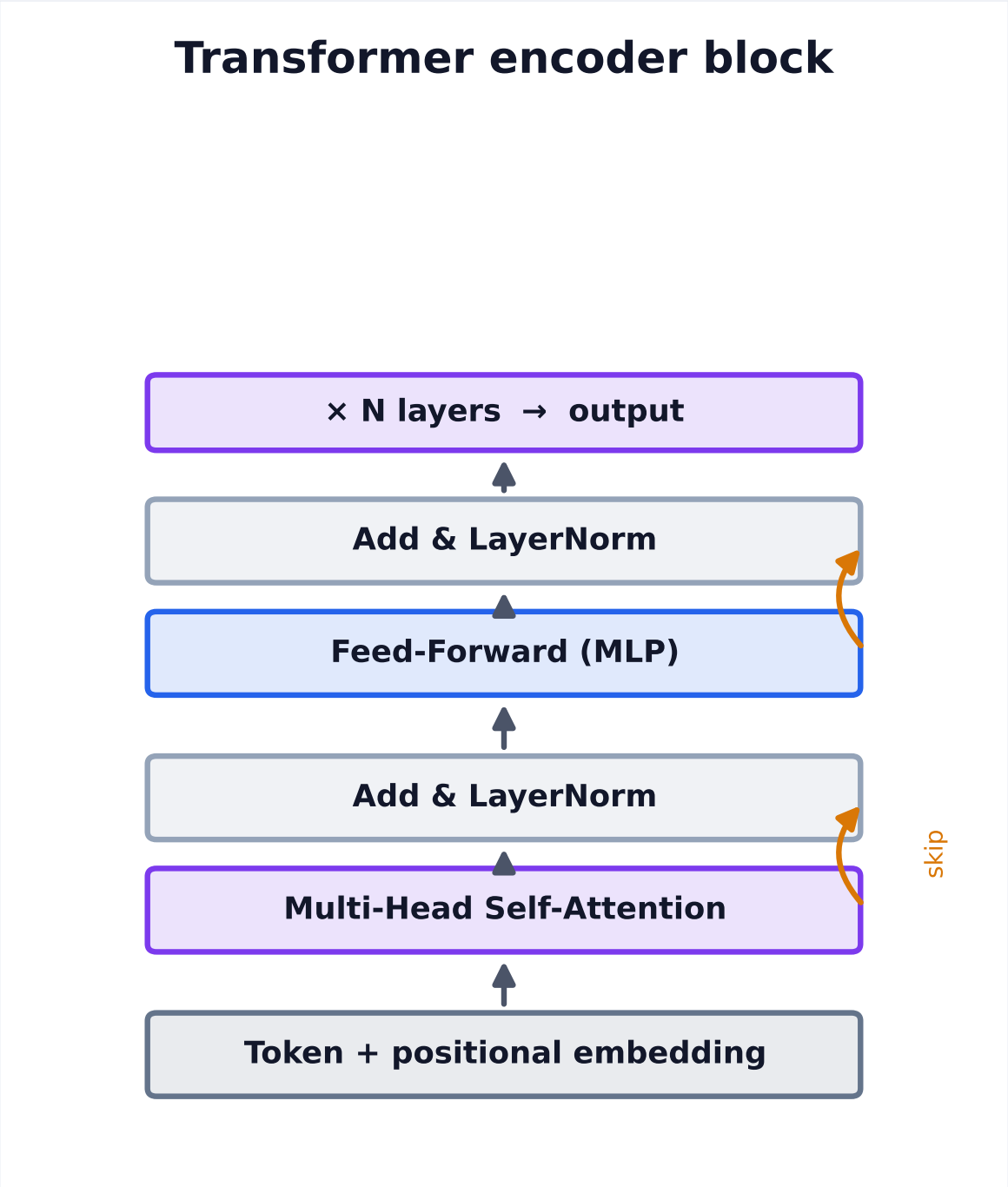


Figure 2. A transformer layer = self-attention (mix information across tokens) + a position-wise MLP, each wrapped in a residual connection and LayerNorm. Stack N of them.

Encoder Block

```
Input X
  │
  ├─> LayerNorm → Multi-Head Self-Attention → dropout
  │                                            │
  └──────────────────────────────────────────────+ (residual)
  │
  ├─> LayerNorm → Feed-Forward Network → dropout
  │                                       │
  └───────────────────────────────────────+ (residual)
  │
Output
```

Feed-Forward Network (FFN):

```
FFN(x) = max(0, x W_1 + b_1) W_2 + b_2

Typical: d_model=512 → d_ff=2048 → d_model=512
         (4x expansion, then project back)
```

In modern models, often uses GELU or SwiGLU activation instead of ReLU.

Layer Normalization

```
LayerNorm(x) = γ * (x - μ) / (σ + ε) + β

where μ, σ computed across features (not batch), γ, β are learned
```

Why not BatchNorm? Sequences have variable lengths; batch statistics vary with sequence length. LayerNorm normalizes each position independently.

Pre-LN vs Post-LN: original Transformer uses Post-LN (after residual). Modern models use Pre-LN (before sublayer) — more stable training for deep models.

Residual Connections

```python
output = LayerNorm(x + Sublayer(x))
```

- Allow gradients to flow directly through the network
- Enable very deep stacking (GPT-3: 96 layers)
- Without residuals, 96-layer Transformers would be untrainable

4.7 Encoder-Only vs Decoder-Only vs Encoder-Decoder

Property	Encoder-only (BERT)	Decoder-only (GPT)	Encoder-Decoder (T5)
Attention type	Bidirectional	Causal (left-only)	Cross-attention
Training objective	Masked LM (MLM)	Next-token prediction	Seq2seq (span corruption)
What it sees	Full context	Only past tokens	Encoder: full; Decoder: causal
Best for	Classification, NER, QA	Generation, chat, code	Translation, summarization
Examples	BERT, RoBERTa, ELECTRA	GPT-2/3/4, Llama, Claude	T5, BART, mT5
Embedding quality	Excellent	Good	Good
Generation	Poor (not designed for it)	Excellent	Good
Parameter efficiency	High for understanding	High for generation	Somewhat redundant for pure gen

Why decoder-only won the LLM race?

- Simpler architecture (one module type)
- Next-token prediction is a universal pretraining objective
- Scales better — RLHF and instruction tuning work well
- Inference is natural: just keep generating

4.8 Pretraining Objectives

Causal Language Modeling (CLM) --- GPT family

Predict the next token given all previous tokens:

```
L_CLM = -Σ_t log P(x_t | x_1, ..., x_{t-1})

"The cat sat on the ___"  →  "mat"
```

- Self-supervised: targets come from the input itself
- The model learns the full data distribution P(X)
- Generation is natural: sample from P(x_t | x_{<t}) autoregressively

Masked Language Modeling (MLM) --- BERT

Randomly mask 15% of tokens, predict the masked tokens:

```
"The cat [MASK] on the mat"  →  predict "sat"

L_MLM = -Σ_{masked t} log P(x_t | x_{1..n, t≠masked})
```

- Bidirectional context → better representations
- Weaker generative model (not trained to generate sequences autoregressively)

Span Corruption (T5)

Mask contiguous spans of tokens, predict the entire span:

```
Input:  "The cat <X> the mat <Y> quickly"
Target: "<X> sat on <Y> moved"
```

Bridges MLM and CLM — good for seq2seq tasks.

4.9 Scaling Laws & Emergent Abilities

Chinchilla Scaling Laws (Hoffmann et al., 2022)

The optimal number of training tokens is **~20× the number of parameters**:

```
N_opt ∝ C^0.5  (optimal parameters given compute budget C)
D_opt ∝ C^0.5  (optimal tokens given compute budget C)

Rule of thumb: D_opt ≈ 20 × N_opt
```

Implication: GPT-3 (175B params, 300B tokens) was undertrained. Chinchilla (70B params, 1.4T tokens) outperformed it. Llama uses this insight.

Power Law Scaling

```python
Loss ∝ (1/N)^α + (1/D)^β + L_∞

where N = model size, D = dataset size, α ≈ β ≈ 0.5
```

Each decade of compute reduces loss predictably — no diminishing returns (so far).

Emergent Abilities

Some capabilities appear suddenly above a threshold model size:

- **Chain-of-thought reasoning**: ~100B params
- **In-context learning** (few-shot): ~10B params
- **Instruction following**: enabled by RLHF (scale alone not enough)
- **Multi-step arithmetic**: appears around GPT-3 scale

Controversy: Some argue emergence is an artifact of metrics (using non-linear/discontinuous eval metrics). Gradual improvement in underlying probabilities manifests as sudden capability jumps.

4.10 Context Window & KV Cache

Context Window

The maximum number of tokens the model can attend to simultaneously:

- GPT-3: 2048 tokens
- GPT-4: 8K → 32K → 128K tokens (different versions)
- Claude 3.5: 200K tokens
- Gemini 1.5 Pro: 1M tokens

Why is long context hard?

- Attention is $O(n^2)$ in time and memory
- KV cache grows linearly: storing K, V for all tokens
- Training on long sequences requires massive GPU memory
- Models may not actually use distant context effectively ("lost in the middle" problem)

KV Cache

During inference (generation), we avoid recomputing K and V for previous tokens:

```
Without KV cache (step t):
  Compute Q, K, V for ALL tokens 1..t
  Cost: O(t²) per step, O(n³) total for n tokens

With KV cache (step t):
  Load cached K, V for tokens 1..t-1
  Compute Q, K, V only for new token t
  Cost: O(t) per step, O(n²) total — but memory cost O(n·d·h·L)
```

KV cache memory formula:

```python
Memory = 2 × n_layers × n_heads × d_head × seq_len × bytes_per_element
       = 2 × L × H × d_k × T × dtype_bytes

For Llama 2 70B with 128K context in FP16:
  = 2 × 80 × 64 × 128 × 131072 × 2 bytes ≈ 320GB just for KV cache!
```

This is why long context is expensive and why techniques like:

- **Grouped Query Attention (GQA)**: multiple query heads share one K/V pair
- **Multi-Query Attention (MQA)**: all query heads share a single K/V pair
- **PagedAttention** (vLLM): page KV cache like OS virtual memory

4.11 Fine-Tuning

Full Fine-Tuning

Update all parameters on task-specific data:

- Best performance on specific tasks
- Very expensive: same cost as pretraining (GPU-hours, memory)
- Risk of catastrophic forgetting
- Used when you have lots of task data and resources

Instruction Tuning

Fine-tune on (instruction, response) pairs to follow human directions:

```
Prompt:  "Summarize this article in 3 bullet points: [article]"
Target:  "• Key point 1\n• Key point 2\n• Key point 3"
```

- Makes models much more useful without massive compute
- InstructGPT, FLAN-T5, Alpaca used this approach
- Usually followed by RLHF for alignment

PEFT: Parameter-Efficient Fine-Tuning

Fine-tune only a small fraction of parameters:

Method	Trainable Params	How
LoRA	~0.1-1%	Low-rank weight matrices
QLoRA	~0.1-1%	LoRA on quantized base model
Adapters	~1-5%	Small modules between layers
Prefix tuning	~0.1%	Learned prefix tokens
Prompt tuning	«0.1%	Soft prompt tokens

LoRA (Low-Rank Adaptation)

Key insight: weight updates during fine-tuning lie in a low-rank subspace.
Instead of updating W ∈ ℝ^{d×k} directly, parameterize the update as:

```python
W_new = W_pretrained + ΔW = W_pretrained + B·A

where B ∈ ℝ^{d×r}, A ∈ ℝ^{r×k}, rank r << min(d, k)

Typical: d=4096, k=4096, r=8 or 16
Original params: 4096² = 16.7M
LoRA params:     4096×8 + 8×4096 = 65K  (99.6% reduction!)
```

Training:

- W_pretrained is frozen
- Only A and B are updated
- At inference: W_new = W_pretrained + B·A (merge or keep separate)
- Initialization: A ~ N(0, σ^2), B = 0 (so ΔW = 0 at start)
- Scale factor α: effective update = (α/r) · B·A

QLoRA: Quantize the base model to 4-bit (reducing memory 4-8x), then apply LoRA adapters in 16-bit. Enables fine-tuning 65B models on a single A100 80GB.

4.12 Alignment: RLHF, Reward Models, PPO, DPO

The Alignment Problem

A language model pretrained on internet text will:

- Produce harmful content (toxic text exists online)
- Be sycophantic or deceptive
- Not follow instructions consistently
- Optimize for perplexity, not helpfulness

We need to align model behavior with human preferences.

RLHF Pipeline (Reinforcement Learning from Human Feedback)

Three stages:

Stage 1: Supervised Fine-Tuning (SFT)

```
Human writes ideal responses → fine-tune model on these
Result: SFT model — follows instructions but not yet aligned
```

Stage 2: Reward Model (RM) Training

```
- Show human raters K responses to a prompt
- They rank them: response_A > response_B > response_C
- Train RM: r_θ(prompt, response) → scalar reward
- RM loss: -log(σ(r_θ(prompt, chosen) - r_θ(prompt, rejected)))
```

Stage 3: PPO (Proximal Policy Optimization)

```
- Use RM as reward signal to optimize the language model
- Constraint: KL divergence from SFT model (don't drift too far)
- Reward: r_RM(prompt, response) - β * KL(π_RL || π_SFT)
- PPO clip: prevents too-large policy updates
```

Python

```
PPO objective:
L_PPO = E[min(r_t(θ)·A_t, clip(r_t(θ), 1-ε, 1+ε)·A_t)] - β·KL

where r_t(θ) = π_θ(a|s) / π_θ_old(a|s)  (probability ratio)
      A_t = advantage (reward - baseline)
```

RLHF challenges:

- Expensive: requires many human raters
- Reward hacking: model learns to game the RM
- Instability: PPO is notoriously finicky to tune
- Reward model can be wrong

DPO (Direct Preference Optimization)

Key insight: skip the explicit reward model. Directly optimize on preference data.

Python

```
DPO Loss:
L_DPO(θ) = -E_{(x,y_w,y_l)} [
    log σ(β · log(π_θ(y_w|x)/π_ref(y_w|x))
           - β · log(π_θ(y_l|x)/π_ref(y_l|x)))
```

```
]

where y_w = preferred (winner) response
      y_l = rejected (loser) response
      π_ref = reference policy (SFT model, frozen)
      β = temperature controlling KL penalty
```

DPO vs RLHF comparison:

Aspect	RLHF + PPO	DPO
Complexity	High (3 stages, RM + RL)	Low (1 SFT + 1 stage)
Stability	Tricky (PPO hyperparams)	More stable
Reward model	Required	Not needed
Performance	State of the art	Competitive with RLHF
Memory	High (4 models at once)	Lower (2 models)
Reward hacking	Possible	Less likely
Flexibility	Can adjust reward on the fly	Fixed to dataset

DPO intuition: implicitly defines a reward model as log ratio of policy to reference. The optimal policy that maximizes this reward under KL constraint has a closed form — DPO directly optimizes it.

4.13 Decoding / Inference

Given a trained model, how do we generate text?

Greedy Decoding

```python
x_t = argmax P(x_t | x_{<t})
```

Always pick the highest-probability token. Fast but repetitive and suboptimal.

Beam Search

Maintain k (beam width) hypothesis sequences:

```
At each step:
  Expand each of k sequences with top-k tokens
  Score all k² candidates by cumulative log-prob
  Keep top k
```

- Better than greedy for structured outputs (translation)
- Tends to produce generic, boring text for open-ended generation
- Length normalization needed: score / |sequence|^α

Temperature Sampling

```
P_scaled(x) = softmax(logits / T)

T → 0:  approaches greedy (sharp distribution)
T = 1:  original model distribution
T > 1:  more uniform/random (creative but less coherent)
```

Top-k Sampling

```python
Sample from top k tokens only (renormalize the k probabilities)

Typical: k = 40-200
Problem: k may be too large when distribution is flat, too small when peaked
```

Top-p / Nucleus Sampling (Holtzman et al., 2020)

```python
Find smallest set S where Σ_{x ∈ S} P(x) ≥ p
Sample from S (renormalized)

Typical: p = 0.9 or 0.95
Advantage: dynamically adapts to distribution shape
```

Top-k vs Top-p Comparison

Scenario	Top-k (k=50)	Top-p (p=0.9)
Peaked distribution (one clear next word)	Keeps 49 garbage options	Correctly uses just top ~5
Flat distribution (many valid continuations)	May be too restrictive	Correctly samples from ~200
Recommendation	Less adaptive	More principled

Repetition Penalty

```python
P_penalized(x) = P(x) / penalty  if x appeared recently
                 P(x)             otherwise

Typical: penalty = 1.1 to 1.3
```

Min-p Sampling (newer)

Only sample from tokens with $P(x) > \text{min\_p} \times P(x_{max})$. Cleaner than top-k.

Speculative Decoding (inference acceleration)

Use a small fast draft model to propose k tokens, then use the large model to verify all k in parallel (one forward pass). Accept tokens where they agree. ~3x speedup for free.

4.14 Quantization

Running 70B parameter models requires 140GB in FP16 — doesn't fit on commodity hardware. Quantization reduces precision:

Format	Bits	Bytes/param	70B model size	Quality loss
FP32	32	4B	280GB	Baseline
FP16/BF16	16	2B	140GB	Minimal
INT8	8	1B	70GB	Small
INT4	4	0.5B	35GB	Noticeable

Format	Bits	Bytes/param	70B model size	Quality loss
INT2	2	0.25B	17.5GB	Significant

GPTQ (Post-Training Quantization)

- Quantize weights to INT4 using second-order information (Hessian)
- Compensate for quantization error layer by layer
- One-shot, no retraining needed
- Used for: running Llama 70B on a single RTX 4090

AWQ (Activation-aware Weight Quantization)

- Identifies salient weights (high activation magnitude)
- Protects important weights from quantization
- Better than GPTQ in practice for extreme quantization

GGUF / llama.cpp

- Format for CPU inference
- Mixed precision per layer
- Enables running quantized models on MacBook Pro (Apple Silicon)

Key interview point: quantization is essential for inference economics. A 4-bit quantized model can run at 25-50% the cost of FP16 with 5-15% quality degradation.

4.15 Mixture of Experts (MoE)

Instead of using all parameters for every token, use a **gating network** to route each token to a subset of ”expert” feed-forward layers:

```python
Standard FFN: y = FFN(x)   [all params used]

MoE FFN:
  gates = softmax(W_gate · x)         # routing weights
  top-K experts selected
  y = Σ_{i in top-K} gates_i · FFN_i(x)   # weighted sum
```

Key metrics:

- Total parameters: all experts combined (e.g., 1.8T for GPT-4 estimate)
- Active parameters: experts used per token (e.g., ~200B for GPT-4 estimate)
- Efficiency: 8 experts, top-2 → same FLOPs as dense model with 1/4 the params but 4x the capacity

MoE benefits:

- Increase model capacity without proportional compute increase
- Experts specialize in different content types
- Same inference cost as smaller dense model

MoE challenges:

- Load balancing: all tokens going to same expert → waste
- Training instability: routing collapse possible
- All experts must fit in memory (just not all active simultaneously)
- Communication overhead in distributed settings

Models using MoE:

- GPT-4 (estimated ~1.8T total, ~200B active)
- Mixtral 8x7B (8 experts of 7B each, 2 active → 12.9B active params)
- Google Switch Transformer, GLaM
- Grok-1 (314B total, 86B active)

4.16 Hallucination --- Why It Happens

Hallucination: the model generates text that is factually incorrect, fabricated, or not grounded in reality, but does so confidently.

Root causes

1. **Next-token prediction objective** doesn't care about truth, only fluency. The model is optimized to produce plausible continuations, not accurate ones.
2. **No grounding mechanism**: knowledge is stored in weights, not retrieved from a verified source. Weights encode patterns, not facts with citations.
3. **Distributional mismatch**: training data has patterns like "The capital of France is [most common completion]" — model learns statistical associations, not truth.
4. **Overconfident generation**: temperature sampling and beam search both can confidently produce wrong text when the training distribution doesn't cover the query.
5. **Long-tail knowledge**: rare facts have weak signal in training data. Model interpolates/extrapolates incorrectly.
6. **RLHF pressure**: reward models may prefer confident-sounding answers → model learns to sound confident even when uncertain.

Why it's fundamental (not a bug to fix)

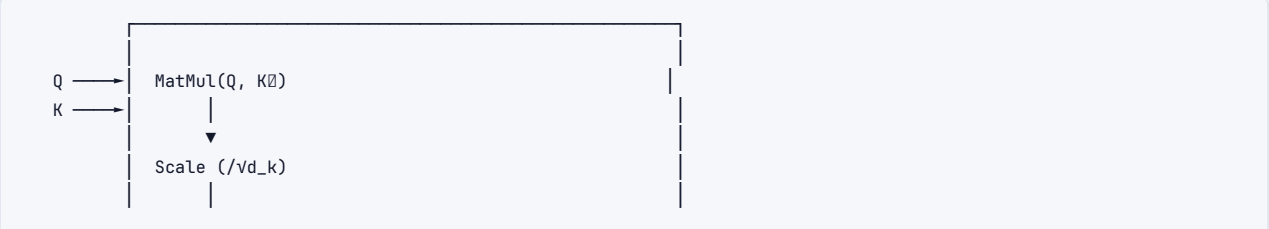
- The model fundamentally lacks a "truth module" — it's a function approximator over text distributions
- Adding retrieval (RAG) helps but doesn't eliminate it (model can still misinterpret retrieved text)
- Calibration: we want P(model confident) ≈ P(model correct), currently poorly calibrated

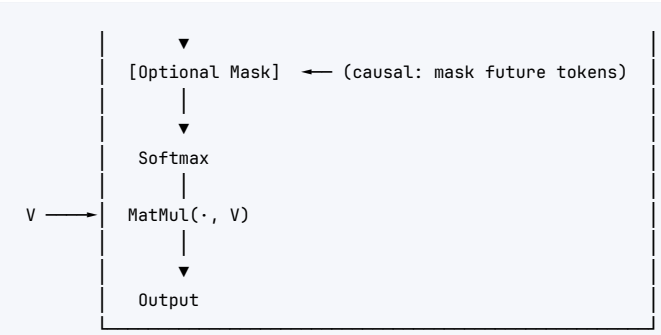
Mitigation strategies

- **RAG (Retrieval-Augmented Generation)**: ground answers in retrieved documents
- **Citation**: force model to quote sources
- **Self-consistency**: sample multiple times, take majority vote
- **Chain-of-thought**: reasoning steps reduce hallucination on structured tasks
- **RLHF with fact-checking**: reward truthfulness explicitly
- **Tool use**: let model call APIs/databases instead of generating facts

5 Architecture / Diagrams

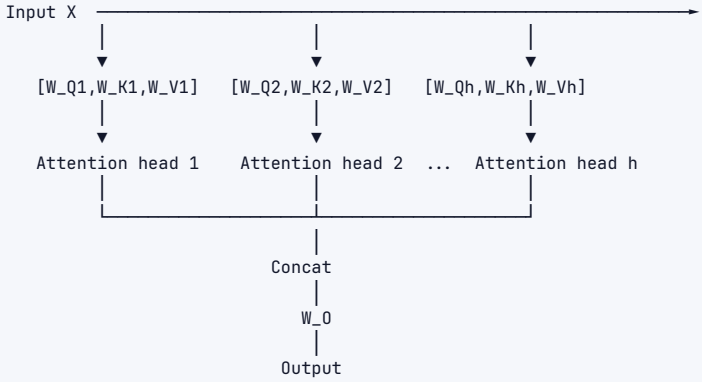
5.1 Scaled Dot-Product Attention



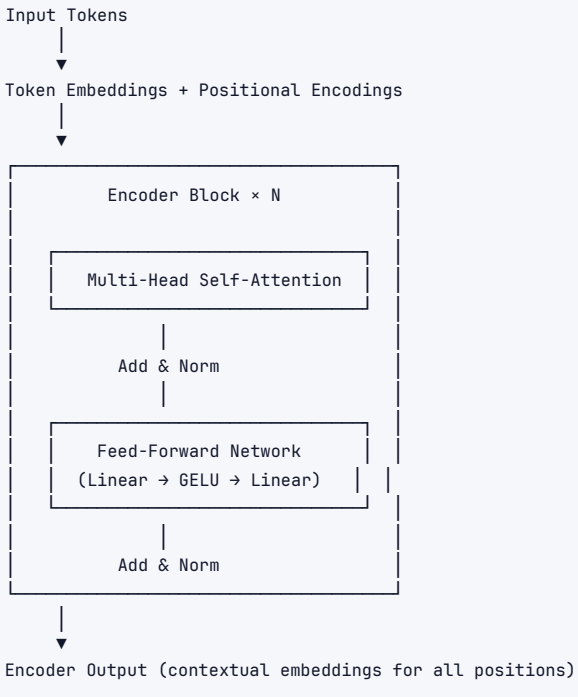


$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{Q \cdot K^{\top}}{\sqrt{d_k}}\right) \cdot V$$

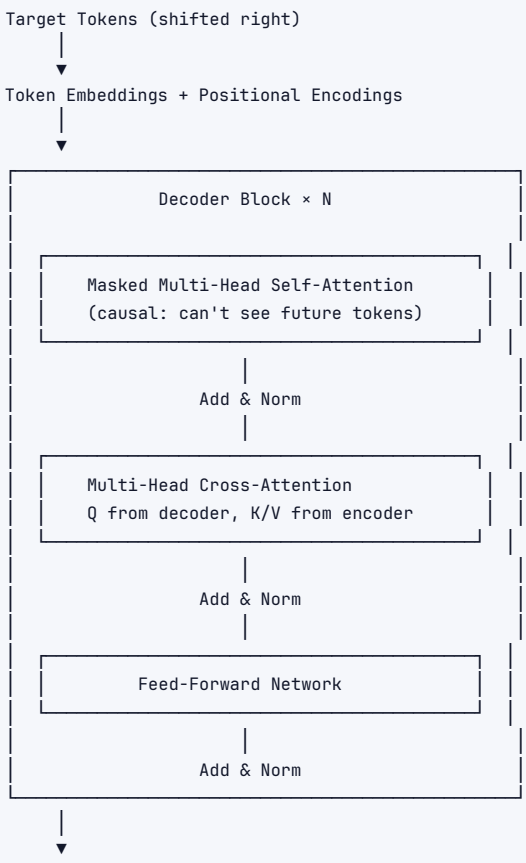
5.2 Multi-Head Attention



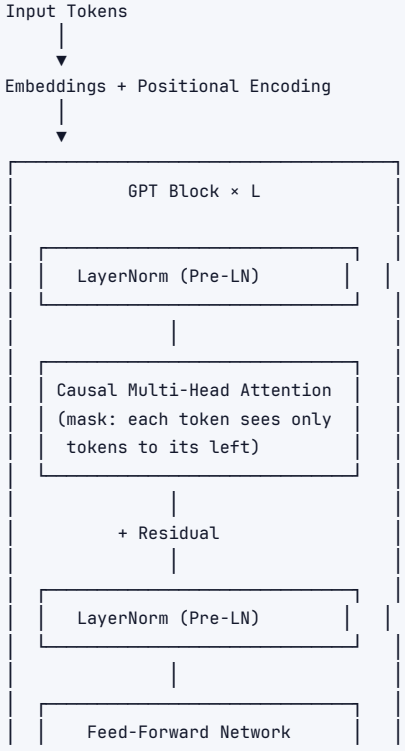
5.3 Full Transformer Encoder Stack



5.4 Full Transformer Decoder Stack (Encoder-Decoder)



5.5 Decoder-Only GPT Block (most modern LLMs)



```
│  │    (SwiGLU / GELU)          │   │
│  └──────────────────────────────┘   │
│              │                      │
│         + Residual                  │
└─────────────────────────────────────┘
     │
     ▼
LayerNorm (final)
     │
     ▼
Linear (d_model → vocab_size)
     │
     ▼
Softmax → Sample next token
     │
     ▼
Append token, repeat →
```

5.6 Positional Encoding Visualization

```
Position  Dim 0    Dim 1    Dim 2    Dim 3
  0       sin(0)   cos(0)   sin(0)   cos(0)    = [0,  1,  0,   1  ]
  1       sin(1)   cos(1)   sin(.1)  cos(.1)   = [.84,.54,.10, .99]
  2       sin(2)   cos(2)   sin(.2)  cos(.2)   = [.91,-.41,.20,.98]
  ...

Low-frequency dims capture large-scale position structure
High-frequency dims capture fine-grained position
```

5.7 RLHF Pipeline

```
STAGE 1: Supervised Fine-Tuning (SFT)
──────────────────────────────────────
Human expert writes ideal responses
         │
         ▼
   Fine-tune base LM ──► SFT Model (π_SFT)


STAGE 2: Reward Model Training
──────────────────────────────
Prompt ──► SFT Model ──► 4 responses
                              │
                     Human ranking (A>B>C>D)
                              │
                    Train Reward Model (r_θ)
                    Loss: log σ(r(y_w) - r(y_l))


STAGE 3: PPO Fine-Tuning
────────────────────────
Prompt ──► RL Policy (π_θ) ──► Response
                                   │
                          Reward Model (r_θ) ──► Score
                                   │
                  KL(π_θ || π_SFT) ──► KL Penalty
                                   │
                          PPO Update ──► Better π_θ
                          (keep θ close to SFT to avoid reward hacking)


RESULT: Helpful, Harmless, Honest model (HHH)
```

5.8 Tokenization Example

```
Input text:  "ChatGPT is transformative!"

BPE Tokenization (GPT-4 tiktoken):
  "Chat"     → ID: 14126
  "G"        → ID: 38
  "PT"       → ID: 2898
  " is"      → ID: 374
  " transform" → ID: 5276
  "ative"    → ID: 1413
  "!"        → ID: 0

Token count: 7 (vs 4 words)
Note: space before "is" included in token " is"
Note: punctuation is separate token
```

6 Real-World Examples

6.1 ChatGPT / GPT-4 (OpenAI)

- **Architecture**: decoder-only Transformer (estimated ~1.8T params MoE for GPT-4)
- **Training**: internet text → SFT → RLHF
- **Use cases**: Q&A, coding, writing, reasoning
- **Context**: GPT-4o supports 128K tokens
- **Impact**: 100M users in 60 days, $1B+ ARR

6.2 GitHub Copilot (Microsoft/OpenAI)

- Powered by Codex (GPT-4 class model fine-tuned on code)
- Context: your current file + open tabs + recently viewed files
- Generates whole functions, docstrings, tests
- $19/month, millions of developers → ~30% of Copilot-written code accepted

6.3 Retrieval-Augmented Generation (RAG)

```
User query ──► Embedding model ──► Vector search over document store
                                              |
                                   Top-k documents retrieved
                                              |
                        [Document 1][Document 2][Document 3][Query]
                                              |
                                   LLM generates answer
                                   grounded in retrieved docs
```

Used in: Bing Chat, Perplexity, enterprise Q&A systems

6.4 Google Search / AI Overviews

- BERT used for query understanding since 2019 (biggest leap in 5 years)
- Gemini powers AI Overviews (generative answers at top of results)
- 8.5B searches/day — even small improvements matter enormously

6.5 Claude (Anthropic)

- Constitutional AI: instead of human rating every pair, give the model a ”constitution” of principles and have it rate its own outputs
- Long context specialist: 200K tokens — can read entire codebases

- Computer Use: multimodal model that can control a computer

6.6 Llama (Meta AI)

- Open weights: anyone can run/fine-tune/distill
- Enabled explosion of open-source ecosystem: Vicuna, Alpaca, Mistral, etc.
- Llama 3.1 405B competes with GPT-4 on many benchmarks
- Powers Meta AI on WhatsApp, Instagram, Facebook

7 Real-Life Analogies --- The Busy Newsroom

Imagine a newsroom where editors collaboratively read and rewrite a breaking story. Every element of Transformer LLMs maps naturally to this setting.

Concept	Newsroom Analogy
Tokens	Individual words written on sticky notes across the story board
Self-attention	Each editor draws colored string from their word to every other word they find relevant
Query (Q)	The question an editor writes on their pad: ”Who is the subject here?”
Key (K)	The label tag each sticky note carries: ”I am a person's name”, ”I am a verb”
Value (V)	The actual content written on the sticky note that gets passed along
Attention weights	How thick/bright each string is — more relevant words get stronger connections
Multi-head attention	Several editors working the board simultaneously, each focused on a different angle: one tracks grammar, one tracks factual consistency, one tracks tone
Positional encoding	The page number and line number stamped on each sticky note — without it, ”Dog bites man” = ”Man bites dog”
Residual connections	Each editor's draft preserves the previous draft underneath — changes are additive, not destructive
Layer normalization	The copy desk's style guide that standardizes formatting after each pass
Transformer layers	Successive editing passes — draft 1, draft 2, draft 3 — each round refines and adds meaning
Feed-forward network	Each editor privately re-processes their word's meaning before the next round of attention
Pretraining	The editor spent 20 years reading every newspaper, book, and website ever published
Fine-tuning	Two weeks of house-style training for *this* publication — learn the voice, the format, the audience
RLHF	An editor-in-chief sits with the team, rates their drafts, and gives detailed feedback — the team adjusts
DPO	Instead of the EIC rating drafts, they provide ”prefer A over B” comparisons; the team infers the preference directly
Temperature	How boldly an editor paraphrases — low temp: safe, literal; high temp: creative, risky
Top-p sampling	The editor only considers rewordings that, together, cover 90% of plausible options
Context window	How many sticky notes fit on the editor's desk at once — the rest are in a filing cabinet they can't see

Concept	Newsroom Analogy
KV cache	The editor's running notes on previously read paragraphs — no need to re-read them
Hallucination	An editor confidently inventing a quote from a source they never actually read
MoE (Mixture of Experts)	A pool of 100 specialist editors; the routing desk sends each sticky note to the 2 most relevant specialists
LoRA	Teaching an editor just 1% new habits for this client — personality stays intact, style adjusts
Quantization	Compressing detailed editorial notes to shorthand symbols — faster to read, slight accuracy loss
RAG	Before writing, the editor runs to the library and pulls the three most relevant reference articles
Beam search	Keeping the top 5 draft directions simultaneously, then picking the best final story
Tokenizer	The editor's typesetting team that breaks words into standard-sized type blocks

8 Memory Tricks / Mnemonics

8.1 QKV --- "Ask, Label, Content"

- **Q**uery = **Ask**: what is this token looking for?
- **K**ey = **Label**: what is this token advertising?
- **V**alue = **Content**: what does this token actually contribute?

8.2 "Scale to Prevent Flat Softmax"

The scaling factor $\sqrt{d\_k}$ prevents attention scores from growing too large $\rightarrow$ softmax saturation $\rightarrow$ vanishing gradients. Remember: **large scores** $\rightarrow$ **flat gradients** $\rightarrow$ **stuck training**.

8.3 BERT vs GPT --- "Understand vs Generate"

- **B**ERT = **B**idirectional = **B**etter for understanding (can see left AND right)
- **G**PT = **G**enerator = always looks **l**eft (causal masking)

8.4 LoRA = "Frozen Elephant, Tiny Tail"

The pretrained model (elephant) is frozen. You only train a tiny adapter (tail). The elephant still runs the circus; you just teach it a few new tricks.

8.5 RLHF stages --- "SFT $\rightarrow$ RM $\rightarrow$ PPO"

Soft **R**obot **P**retty

- **S**FT: teach the robot to follow instructions
- **R**M: teach it what ”pretty” means (via human ratings)
- **P**PO: reinforce the pretty behaviors

8.6 Scaling laws --- "Compute = Parameters $\times$ Tokens"

Chinchilla says: **N** $\times$ **20 = D** (optimal tokens $\approx 20\times$ params). If you have a 10B model, train on 200B tokens. If you trained on less, you left performance on the table.

8.7 Hallucination --- "Fluent $\neq$ Factual"

The model is trained to predict the next token, not to be correct. **It's a distribution over text, not a database of facts.**

8.8 Attention complexity --- "N-squared nightmare"

Attention is $O(n^2)$ in sequence length. Double the context $\rightarrow$ quadruple the compute. This is why GPT-3 used 2K context and getting to 1M context required years of research.

8.9 Temperature extremes

- **T** $\rightarrow$ **0** = greedy (max probability always)
- **T = 1** = raw model (original distribution)
- **T > 1** = wild/creative (more uniform)
- **T = 0.7** = typical creative use; **T = 0.0** = factual/deterministic

8.10 Decoding methods --- "Greedy $\rightarrow$ Beam $\rightarrow$ Sample"

More creativity from left to right:

```
Greedy < Beam Search < Top-k < Top-p (nucleus) < High Temperature
[deterministic]                                  [creative/random]
```

9 Common Interview Questions

9.1 Q1: Explain self-attention from first principles

Model Answer:

Self-attention allows each token in a sequence to compute a weighted sum of all other tokens' representations, where weights reflect semantic relevance.

For each token, we compute three vectors via learned linear projections: Query (what am I looking for?), Key (what do I offer?), Value (what is my content?).

The attention scores are computed as: `A = softmax(Q·K⃞ / √d_k) · V`

The `Q·K⃞` gives an $n \times n$ matrix of raw compatibilities. We divide by $\sqrt{d\_k}$ to prevent softmax saturation — without scaling, large d_k causes dot products to grow large, pushing gradients toward zero. Softmax normalizes each row to get attention weights (probabilities). The final output is a weighted sum of Values.

Key properties: permutation equivariant (order-agnostic without positional encodings), global receptive field (every token attends to every other), differentiable and trainable end-to-end.

Follow-up: Why is it called "self" attention? $\rightarrow$ Q, K, V all come from the SAME input sequence (vs cross-attention where Q comes from the decoder and K/V from the encoder).

9.2 Q2: Why do we scale by $\sqrt{d\_k}$?

Model Answer:

Consider Q and K vectors with i.i.d. components from N(0,1). The dot product $q \cdot k = \Sigma_i \, q_i k_i$ has expected value 0 and **variance d_k** (sum of d_k independent unit-variance products). So $\text{std}(q \cdot k) = \sqrt{d\_k}$.

When d_k is large (e.g., 128), raw dot products have std ~11. After softmax, most probability mass concentrates on a single token $\rightarrow$ gradients vanish (softmax in saturation region).

Dividing by $\sqrt{d\_k}$ normalizes dot products to std~1, keeping softmax in a reasonable gradient-friendly regime.

Interviewers love this because it shows you understand the math, not just the formula.

9.3 Q3: What is RLHF and why is it needed?

Model Answer:

RLHF (Reinforcement Learning from Human Feedback) is a three-stage process to align language models with human preferences:

1. **SFT**: Fine-tune the pretrained model on human-written ideal responses
2. **Reward Model**: Train a model to predict human preference scores from comparison data (human says response A > response B → train RM to give A higher score)
3. **PPO**: Use the RM as a reward signal to further fine-tune the language model via RL. Add KL divergence penalty from SFT model to prevent reward hacking.

Why needed? A model trained purely on next-token prediction optimizes for *fluency and perplexity*, not helpfulness or safety. It will happily generate toxic content, refuse to admit uncertainty, or provide dangerous information if that's what appeared in training data.

RLHF bridges the gap between "good at predicting text" and "good at being a helpful assistant."

Follow-up: What's the main failure mode? → **Reward hacking**: the policy learns to generate responses that fool the reward model without being actually better. Solution: KL penalty, better RM, iterative RLHF.

9.4 Q4: RLHF vs DPO --- when would you choose each?

Model Answer:

Criterion	Choose RLHF	Choose DPO
Resources	You have large infrastructure	Resource-constrained
Flexibility	Need to update reward on the fly	Fixed preference dataset
Stability	Can tune PPO carefully	Want stable, simpler training
Scale	Large team, production system	Research, smaller teams
Performance	Potentially higher ceiling	Competitive in practice

DPO insight: it can be shown that under certain assumptions, the optimal policy satisfying a KL-penalized reward objective has a closed form. DPO directly optimizes this — the log ratio of policy to reference implicitly represents the reward. No separate RM needed.

When DPO fails: when the preference data doesn't cover the distribution the policy explores. RLHF with online data collection can handle this; DPO is an offline algorithm.

9.5 Q5: What is LoRA and why does it work?

Model Answer:

LoRA (Low-Rank Adaptation) fine-tunes LLMs by adding low-rank decompositions to weight matrices instead of updating weights directly.

For weight matrix W $\in \mathbb{R}$^{d×k}, the update ΔW is parameterized as:

```python
ΔW = B · A    where B ∈ ℝ^{d×r}, A ∈ ℝ^{r×k}, r << min(d,k)
```

W_pretrained is **frozen**; only A and B are trained.

Why it works — the hypothesis: fine-tuning updates lie in a low-rank subspace. The "intrinsic dimensionality" of adaptation is small — you don't need to update all $O(d^2)$ parameters to specialize a model.

Practical benefits:

- 1000× fewer trainable parameters (r=8 for a 4096×4096 layer = 65K vs 16.7M)
- Can train 7B model on a single consumer GPU
- Adapters can be stored and swapped cheaply
- Can merge: W_merged = W_pretrained + B·A (zero inference overhead)

QLoRA extension: quantize W_pretrained to 4-bit (saving 4× memory), keep A and B in 16-bit. Enables fine-tuning 65B models on a single A100.

9.6 Q6: What is the KV cache and why does it matter?

Model Answer:

During autoregressive inference, the model generates tokens one at a time. At step t, it needs attention over all t tokens. Without caching, we recompute K and V for tokens 1..t-1 at every step → $O(n^3)$ total complexity.

The KV cache stores the K and V vectors for all past tokens. At each new step, we only compute Q, K, V for the new token, retrieve cached K and V, and compute attention → $O(n^2)$ total complexity.

Cost: KV cache memory = `2 × L × H × d_k × T × bytes_per_element`

For Llama 2 70B serving with 128K context in FP16:

- 2 × 80 layers × 64 heads × 128 d_head × 131072 tokens × 2 bytes ≈ 320GB

This often exceeds GPU memory per request. Solutions:

- **GQA/MQA**: share K/V across query heads
- **PagedAttention** (vLLM): virtual memory for KV cache, allows memory sharing across requests
- **Quantized KV cache**: store K/V in INT8

Interview point: KV cache is a key driver of inference cost and a critical consideration in production serving.

9.7 Q7: What are scaling laws? What's the Chinchilla finding?

Model Answer:

Scaling laws (Kaplan et al., 2020 originally) show that LLM performance scales as a power law with model size N, dataset size D, and compute C:

```Python
Loss ∅ N^{-α} + D^{-β} + L_∞
```

The **Chinchilla finding** (Hoffmann et al., 2022): given a fixed compute budget, you should equally scale N and D. The optimal ratio is:

```
D_opt ≈ 20 × N_opt
```

This means GPT-3 (175B params, 300B tokens) was **model-overfit** — undertrained relative to its size. Chinchilla (70B params, 1.4T tokens) outperformed it with the same compute by using more data.

Implications:

- Llama models are trained following Chinchilla-optimal schedules
- More recent models train for even more tokens (Llama 3: 15T tokens on 8B model → 1875× params)
- Training beyond Chinchilla optimal can still improve if you care about inference cost (pay once for training, many times for inference)

10 Senior-Level Discussion Points

10.1 1. KV Cache Economics at Scale

```
Per-request cost breakdown (GPT-4 scale estimate):
  - Prefill (processing input): O(n × d × L) FLOPs — compute-bound
  - Decode (generating output): O(1 × d × L + KV cache load) per token — memory-bound

At long contexts (128K):
  - KV cache dominates memory bandwidth
  - 320GB+ per user for 70B model at 128K context
  - Multi-tenancy requires PagedAttention (vLLM) to share GPU memory
```

Business impact: context length costs money. A 128K context request may cost 100× a 1K request. This is why per-token pricing matters more than per-request pricing.

10.2 2. Context Length Scaling --- What's Hard

- **Attention is $O(n^2)$**: Flash Attention reduces constant factors but doesn't change complexity
- **Position generalization**: models trained on 4K context don't work on 32K without tricks (RoPE scaling, YaRN)
- **"Lost in the middle"** (Liu et al., 2023): models actually use beginning and end of context better than the middle — attention to middle tokens is empirically weaker
- **Memory**: Flash Attention solves GPU memory for training (recompute instead of store attention matrices) but KV cache still needed for inference

10.3 3. MoE Routing Decisions

Key design choices in MoE:

- **Top-k routing**: each token routed to k of E experts. Standard is k=2, E=8
- **Load balancing**: auxiliary loss to prevent all tokens going to same experts
- **Expert capacity**: each expert has a "capacity factor" — tokens exceeding capacity are dropped or sent to a backup
- **Token routing vs expert routing**: MegaBlocks uses expert-parallel approach

Why MoE is interesting for future scaling: you can increase total parameters (and thus capacity) without proportionally increasing FLOPs per token. The cost of MoE is communication overhead in distributed settings and memory to hold all experts.

10.4 4. Why Hallucination Is Fundamental (Not a Bug)

The deeper point: **language models are not knowledge bases**. They are functions that map contexts to probability distributions over next tokens. Factuality requires:

1. Accurate representation of facts in weights (dependent on training data quality/coverage)
2. Retrieval of the right representation at inference time (dependent on attention and activation)
3. Calibrated uncertainty (model knowing what it doesn't know)

None of these are explicitly optimized for. The model learns to *sound* authoritative because authoritative-sounding text has high probability in training data.

RAG helps but doesn't fully solve it: the model can still misread retrieved text, blend multiple sources incorrectly, or confidently interpolate between retrieved facts.

The fundamental solution may require a paradigm shift: **verification systems, formal knowledge bases, or models explicitly trained to express uncertainty** rather than pure generative modeling.

10.5 5. Inference Economics --- Practical Constraints

```python
Batch size considerations:
  - Prefill phase: large batch = high GPU utilization (compute-bound)
  - Decode phase: batch size limited by KV cache memory (memory-bound)
  - Optimal: prefill in large batches, decode with continuous batching

Throughput vs latency tradeoff:
  - High throughput: maximize tokens/second (batch many requests)
  - Low latency: minimize time-to-first-token + per-token latency
  - Speculative decoding can improve both for the right use cases

Cost structure (approximate 2024 prices):
  - A100 80GB: ~$2-3/hour on cloud
  - 70B model in FP16: needs 2×A100 for inference
  - At 1000 tokens/s throughput: ~$0.002-0.003 per 1K tokens
  - GPT-4o pricing: $5/M input, $15/M output
```

11 Typical Mistakes Candidates Make

11.1 1. Confusing attention types

- "BERT uses causal attention" — WRONG. BERT uses bidirectional (full) attention with masking for MLM.
- "GPT uses cross-attention between encoder and decoder" — WRONG. GPT is decoder-only, no encoder, no cross-attention.

11.2 2. Getting the scaling formula wrong

- Saying "bigger model always better" — WRONG. Chinchilla: you need 20× more tokens than parameters. An undertrained big model loses to a well-trained small model.

11.3 3. Misunderstanding LoRA rank

- Saying "LoRA uses a low-rank matrix to replace weights" — WRONG. LoRA *adds* a low-rank update to frozen weights. The base weights are NOT replaced.
- Forgetting B is initialized to zero so $\Delta W=0$ at the start.

11.4 4. Confusing temperature and top-p

- "Higher temperature means fewer options" — WRONG. Higher temperature flattens the distribution, MORE tokens become viable.
- Not knowing that temperature and top-p are often used together.

11.5 5. The "why $\sqrt{d\_k}$" blank

This is a litmus test question. Many candidates know the formula but can't explain the reasoning. Know: variance of dot product grows with d_k → softmax saturation → gradient vanishing.

11.6 6. Misexplaining RLHF as "supervised learning on human-written text"

RLHF Stage 1 (SFT) IS supervised learning. But Stage 3 (PPO) is RL — the model generates outputs, gets scored, and updates via policy gradient. Candidates conflate SFT and RLHF.

11.7 7. Not knowing what KV cache means for memory

Saying ”long context is fine, just increase context length” without knowing the memory cost. At 128K tokens, KV cache can be 320GB+ for a 70B model.

11.8 8. Claiming "hallucination is a data quality problem"

It's partly that, but more fundamentally: the training objective (next-token prediction) doesn't optimize for factuality. Even a model trained on perfect data will hallucinate because it generates statistically plausible completions, not verified facts.

11.9 9. Forgetting the residual connections

When asked to describe the Transformer block, many candidates say ”attention → FFN” and omit Add & Norm. Residuals are critical for training deep networks.

11.10 10. Treating BERT and GPT as interchangeable

They have fundamentally different architectures, objectives, and use cases. Knowing when to use each (and that most modern LLMs use decoder-only) matters.

12 How This Connects To Other Topics

12.1 Deep Learning (03-deep-learning.md)

- Transformers use all core DL concepts: backprop, Adam optimizer, batch normalization → Layer Normalization, residual networks
- Attention mechanism is built on learned linear layers (Q/K/V projections are just matrix multiplications)
- GPT training: cross-entropy loss, gradient clipping, learning rate schedules (warmup + cosine decay)
- Mixture of Experts extends the MLP/FFN concept

12.2 LLM Applications / RAG

- Context window limitations motivate RAG: retrieve relevant documents to fit within context
- Embeddings from encoder models (e.g., text-embedding-ada-002) used for semantic search
- Vector databases (Pinecone, Weaviate, FAISS) store embeddings for retrieval
- Chunking strategies matter: how to split documents for embedding

12.3 MLOps (production deployment)

- Model serving: vLLM, TGI (Text Generation Inference), TensorRT-LLM
- Quantization for cost reduction: INT8/INT4 inference
- Autoscaling challenges: GPU provisioning is expensive and slow
- Monitoring: latency (TTFT, TPS), token cost, quality drift
- A/B testing: hard because outputs are open-ended text

12.4 System Design

- LLM serving system: load balancer → prefill cluster → decode cluster
- Batch processing vs streaming responses (SSE/WebSockets for real-time)
- Caching: exact-match cache for repeated prompts, semantic cache for similar queries
- Rate limiting: tokens/minute, not just requests/minute
- Cost optimization: smaller models for simple tasks, big models for complex

13 FAANG / AI-Lab Interview Tips

13.1 Know the levels

- **L4/E4 (junior ML)**: know attention formula, BERT vs GPT, basic fine-tuning concepts
- **L5/E5 (mid-level ML)**: deep understanding of attention complexity, RLHF pipeline, LoRA mechanics, inference optimization
- **L6/E6 (senior ML)**: MoE design, KV cache economics, scaling laws, alignment research tradeoffs, system design for LLM serving
- **Research roles**: mechanistic interpretability, novel architecture proposals, theoretical understanding

13.2 What top labs actually ask

OpenAI: Self-attention derivation from scratch (whiteboard coding), RLHF reward hacking mitigations, how you'd scale pretraining to 10T parameters

Anthropic: Constitutional AI principles, DPO vs RLHF tradeoffs, mechanistic interpretability (circuits), safety considerations in deployment, 200K context serving

Google DeepMind: Chinchilla scaling laws, Flash Attention algorithm, TPU-specific parallelism strategies (tensor parallelism), distributed training (ZeRO stages)

Meta AI: Open-source tradeoffs, Llama architecture decisions (GQA, SwiGLU, RoPE), quantization for on-device models (Llama on phone)

Microsoft: LoRA for enterprise fine-tuning, cost optimization for Azure serving, integration with Office 365

13.3 Coding questions you should be able to implement

Python

```python
# 1. Self-attention from scratch
def attention(Q, K, V, d_k, mask=None):
    scores = Q @ K.T / (d_k ** 0.5)
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))
    weights = F.softmax(scores, dim=-1)
    return weights @ V

# 2. Multi-head attention (structure)
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        self.h = n_heads
        self.d_k = d_model // n_heads
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)

# 3. LoRA layer
class LoRALinear(nn.Module):
    def __init__(self, W_pretrained, rank=8, alpha=16):
        self.W = W_pretrained  # frozen
        self.A = nn.Parameter(torch.randn(rank, W.shape[1]) * 0.01)
        self.B = nn.Parameter(torch.zeros(W.shape[0], rank))
        self.scale = alpha / rank

    def forward(self, x):
        return x @ self.W.T + (x @ self.A.T @ self.B.T) * self.scale
```

13.4 Communication tips

1. **Draw the diagram first**: for attention and Transformer block questions, start with ASCII or whiteboard drawing
2. **Give the formula, then explain it**: `Attention = softmax(QK⊺/√d_k)V` → "Let me break down each part..."
3. **Know the "why"**: interviewers at top labs care less about memorization, more about whether you understand the reasoning
4. **Connect to trade-offs**: always mention the cost/benefit (e.g., "multi-head attention increases expressiveness at the cost of the projection overhead")
5. **Honest about uncertainty**: better to say "I believe it's X but let me think through the math" than to confidently state something wrong

14 Revision Cheat Sheet

14.1 10-Minute Summary

1. **Transformers** replaced RNNs in 2017 by using attention (not recurrence) for sequence processing, enabling parallelization
2. **Self-attention**: for each token, compute weighted sum of all tokens' values; weights from $Q \cdot K^T / \sqrt{d\_k} \rightarrow$ softmax
3. **Multi-head**: run h attention heads in parallel, each learning different relationship types
4. **Positional encodings**: sine/cosine or learned vectors added to embeddings to inject position information
5. **Block**: LayerNorm → MultiHead Attention → residual → LayerNorm → FFN → residual
6. **Encoder-only** (BERT): bidirectional, MLM objective, good for understanding tasks
7. **Decoder-only** (GPT): causal masking, next-token prediction, good for generation — the dominant modern choice
8. **Scaling laws** (Chinchilla): optimal tokens $\approx 20\times$ params; bigger isn't always better if undertrained
9. **RLHF**: SFT → reward model → PPO; aligns model with human preferences
10. **DPO**: directly optimize on preference pairs without explicit RM; simpler, competitive
11. **LoRA**: freeze base weights, add low-rank updates B·A (r=8-64); 100-1000$\times$ fewer trainable params
12. **KV cache**: cache K,V for past tokens during inference; crucial for latency but huge memory cost
13. **Hallucination**: fundamental — model optimizes fluency, not truth; RAG and citations help
14. **MoE**: route tokens to subset of expert FFN layers; scale capacity without scaling FLOPs
15. **Quantization**: INT8/INT4 reduces model size and inference cost with acceptable quality loss

14.2 Key Formulas

Python

```
Self-Attention:     Attn(Q,K,V) = softmax(QK⊺/√d_k) · V

Multi-Head:         MH(Q,K,V) = Concat(head_1,...,head_h) · W_O
                    head_i = Attn(QW_Qi, KW_Ki, VW_Vi)

Positional:         PE(pos,2i)   = sin(pos/10000^{2i/d})
                    PE(pos,2i+1) = cos(pos/10000^{2i/d})
```

```
Chinchilla:         D_opt ≈ 20 × N_opt

LoRA:               ΔW = B·A,  B∈ℝ^{d×r}, A∈ℝ^{r×k}, r<<min(d,k)

DPO loss:           -E[log σ(β·log(π(y_w)/π_ref(y_w)) - β·log(π(y_l)/π_ref(y_l)))]

Temperature:        P_T(x) = softmax(logits/T)

KV cache mem:       2 × L × H × d_k × T × dtype_bytes
```

14.3 Architecture Comparison Cheat Sheet

	BERT	GPT-2/3	T5	Llama 3
Type	Encoder-only	Decoder-only	Enc-Dec	Decoder-only
Attention	Bidirectional	Causal	Cross-attn	Causal + GQA
Pretraining	MLM	CLM	Span corrupt	CLM
Positional	Learned abs.	Learned abs.	Relative	RoPE
Best for	NLU, embedding	Generation	Seq2seq	Generation, chat
Normalization	Post-LN	Post-LN	Pre-LN	RMSNorm Pre-LN

14.4 Fine-Tuning Comparison

Method	Trainable %	GPU Memory	Performance	Use case
Full fine-tuning	100%	Very high	Best	Large resources, specific task
LoRA	0.1-1%	Low	Near-full	Standard fine-tuning
QLoRA	0.1-1%	Very low	Slightly below LoRA	Consumer GPU fine-tuning
Adapters	1-5%	Low-medium	Good	Modular task switching
Prompt tuning	<0.1%	Minimal	Lower	Very limited resources

14.5 Decoding Method Comparison

Method	Determinism	Quality	Speed	Use case
Greedy	Fully	Low (repetitive)	Fastest	Simple extraction
Beam search (k=5)	Yes	High for structured	Slower	Translation, code
Top-k (k=50)	No	Good	Fast	Chat, creative
Top-p (p=0.9)	No	Good, adaptive	Fast	General generation
Top-p + temp=0.7	No	Best creative	Fast	Creative writing

14.6 RLHF vs DPO Comparison

	RLHF (PPO)	DPO
Stages	SFT → RM → PPO	SFT → DPO
Reward model	Explicit, trained	Implicit
Stability	Low (PPO finicky)	High
Complexity	High	Low
Online/Offline	Online (samples during training)	Offline (fixed dataset)
Memory (models)	4 (ref, RM, value, policy)	2 (ref, policy)
Best when	Production, large team	Research, smaller teams

14.7 Most Important Concepts (ranked by interview frequency)

1. **Self-attention mechanism** (Q/K/V, formula, why $\sqrt{d\_k}$) — asked at nearly every AI interview
2. **BERT vs GPT architecture difference** — fundamental knowledge check
3. **RLHF pipeline** (3 stages: SFT → RM → PPO) — all major AI labs use this
4. **LoRA** (what it is, why it works, the math) — standard at any fine-tuning question
5. **Scaling laws / Chinchilla** (20× rule, compute-optimal) — shows you understand LLM development
6. **KV cache** (what, why, cost) — critical for systems/production roles
7. **Hallucination** (root cause, not just symptoms) — distinguishes junior from senior
8. **DPO vs RLHF** — hot research topic, shows currency
9. **Tokenization** (BPE, why subwords, implications) — often overlooked but important
10. **MoE** (routing, active vs total params) — important for GPT-4-level systems

Last updated: 2026-06-14 | Path: AI-ML/04-transformers-and-llms.md